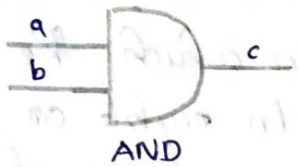
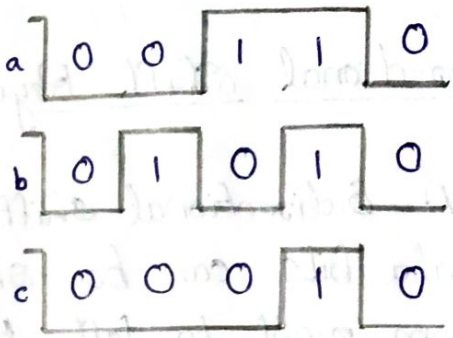
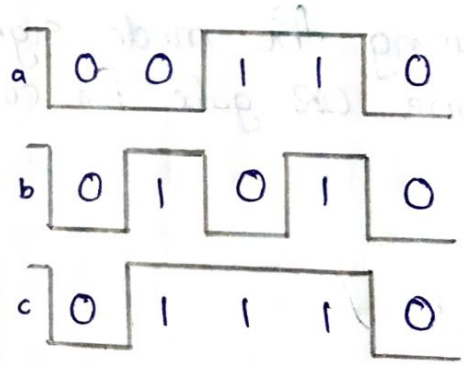
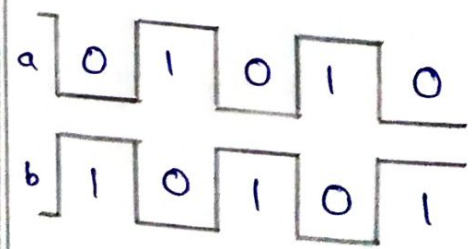
GATE LOGIC SYMBOL	TRUTH TABLE	WAVEFORMS
 AND		
 OR		
 NOT		

a	b	c
0	0	0
0	1	0
1	0	0
1	1	1

a	b	c
0	0	0
0	1	1
1	0	1
1	1	1

a	b
0	1
1	0

08/08/22

EXPERIMENT-10

DEVELOPMENT OF VERILOG MODULES FOR LOGIC GATES

Aim : Develop verilog modules for logic gates and verify the truth table.

Programs (Structural Modeling):

1. AND GATE

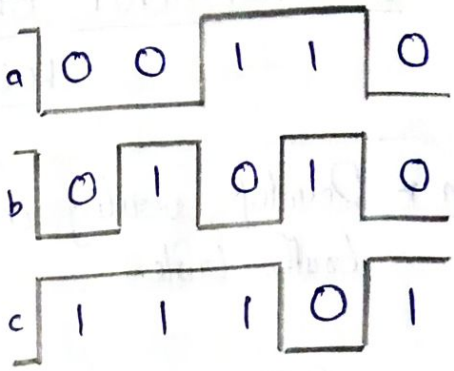
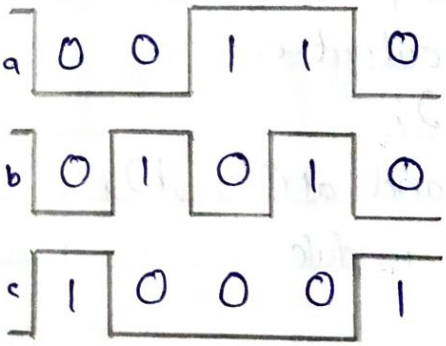
```
module andgate (
    input a, b,
    output c
    );
    and a1 (c, a, b);
end module
```

2. OR GATE

```
module orgate()
    input a, b,
    output c
    );
    or o1 (c, a, b);
end module
```

3. NOT GATE

```
module notgate( input a, output b );
    not n1 (b, a);
end module
```

GATE LOGIC SYMBOL	TRUTH TABLE	WAVEFORMS
 NAND		
 NOR		

a	b	c
0	0	1
0	1	1
1	0	1
1	1	0

a	b	c
0	0	1
0	1	0
1	0	0
1	1	0

4, NAND GATE

```
module nandgate ( input a,b, output c);
    nand n1 (c, a, b);
end module
```

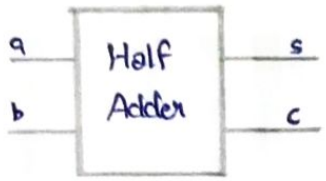
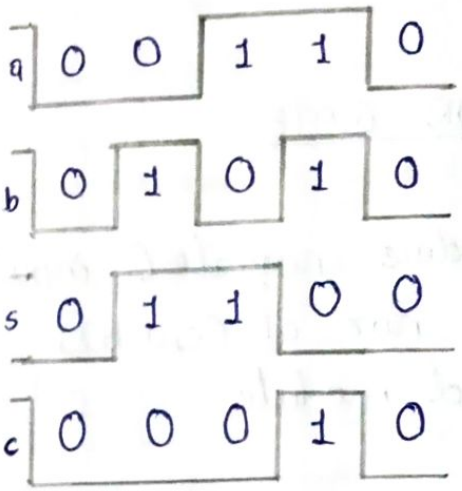
5) NOR GATE

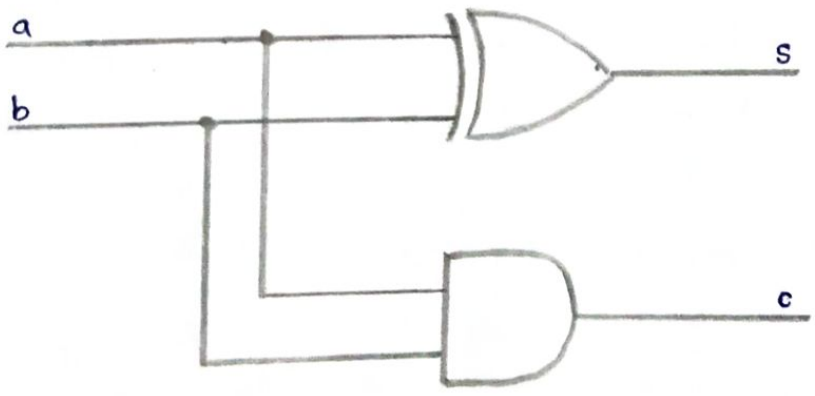
```
module norgate ( input a,b, output c);
    nor o1 (c,a,b);
end module
```

8/8/22

Result :

Developed verilog modules for logic gates and verified the truth table.

LOGIC SYMBOL	TRUTH TABLE	WAVEFORMS
 HALF ADDER	a b s c 0 0 0 0 0 1 1 0 1 0 1 0 1 1 0 1	



LOGIC DIAGRAM

08/08/22

EXPERIMENT-11

DEVELOPMENT OF VERILOG MODULES FOR HALF ADDER AND FULL ADDER

Aim: Develop a verilog program code to implement half adder using 3 types of modelling. and full adder using two half adders.

1. HALF ADDER (DATA FLOW MODELING)

```
module has (input a,b, output c,s);
   assign {c,s} = a+b;
end module
```

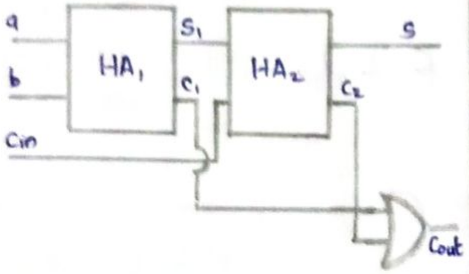
2. HALF ADDER (STRUCTURAL MODELING)

```
module hastut (input a,b, output s,c);
   xor n1 (s,a,b);
   and n2 (c,a,b);
endmodule
```

3. HALF ADDER (BEHAVIOURAL MODELING)

```
module habeh (input a,b, output reg s,c);
   always @(a,b)
   {c,s} = a+b;
end module
```

LOGIC SYMBOL

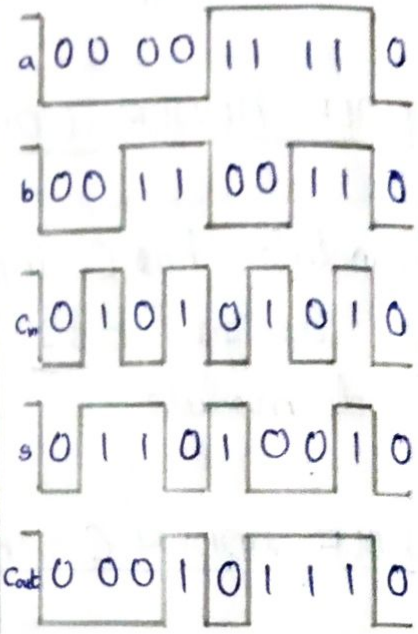


FULL ADDER

TRUTH TABLE

a	b	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

WAVEFORMS



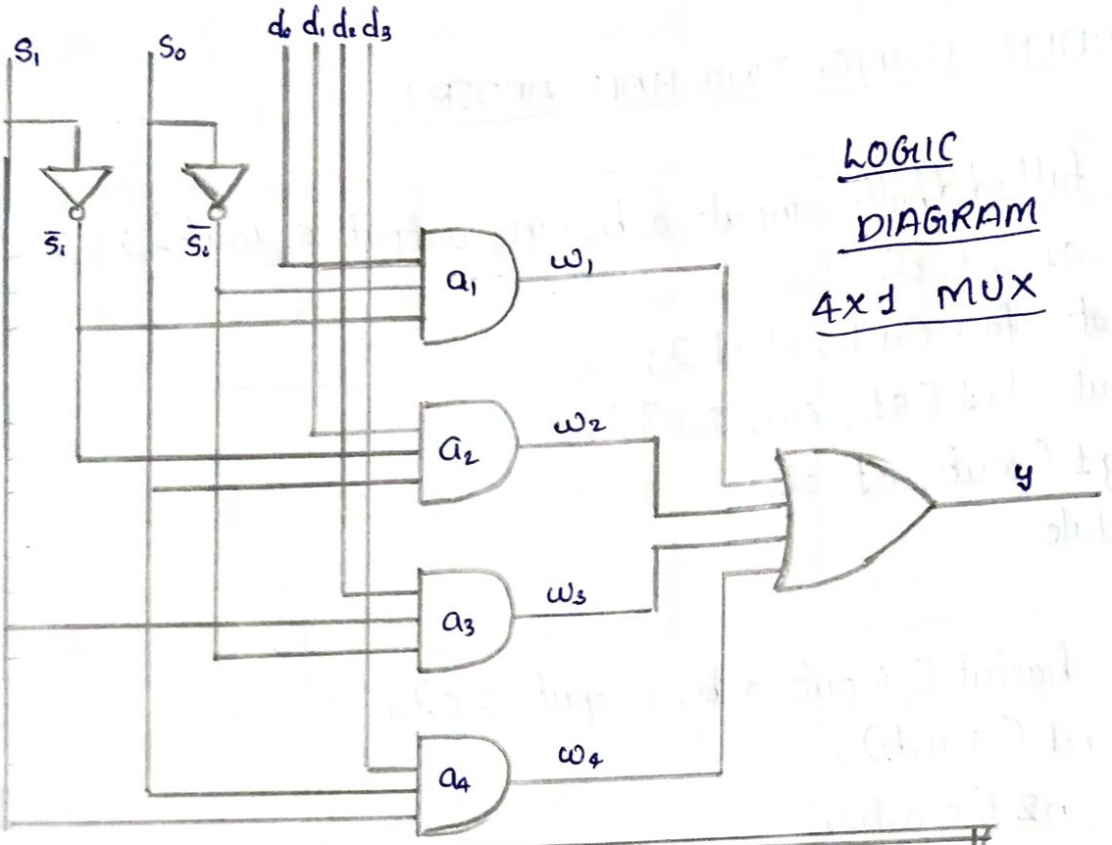
4. FULL ADDER USING TWO HALF ADDERS

```
module fulladdhalf ( input a,b, cin, output s, cout);
   wire s1, c1, c2;
   hastut fa0 (a,b, s1, c1 );
   hastut fa1 (s1, cin, s, c2 );
   or g1 (cout, c1, c2);
end module

module hastut ( input a,b, output s,c);
   xor n1 (s,a,b);
   and n2 (c,a,b);
endmodule
```

Result :-

The development of verilog program code to implement half adder and full adder using two half adders were done.

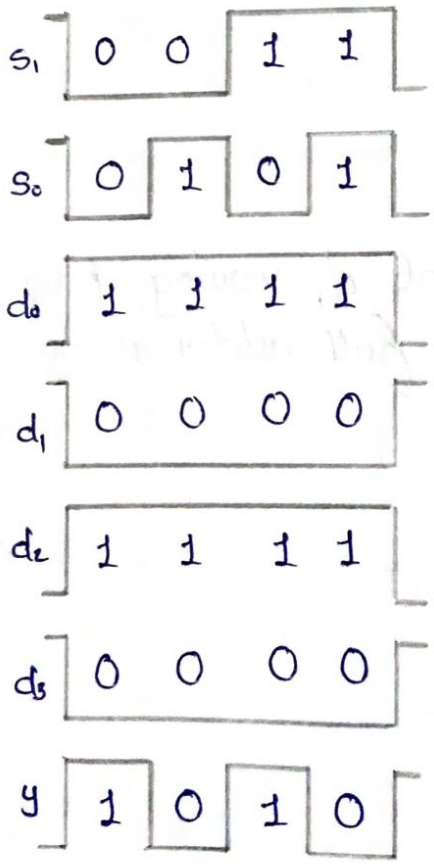


LOGIC DIAGRAM 4X1 MUX

Truth Table:

input	S_1	S_0	y
d_0	0	0	d_0
d_1	0	1	d_1
d_2	1	0	d_2
d_3	1	1	d_3

Waveforms:



08/08/22

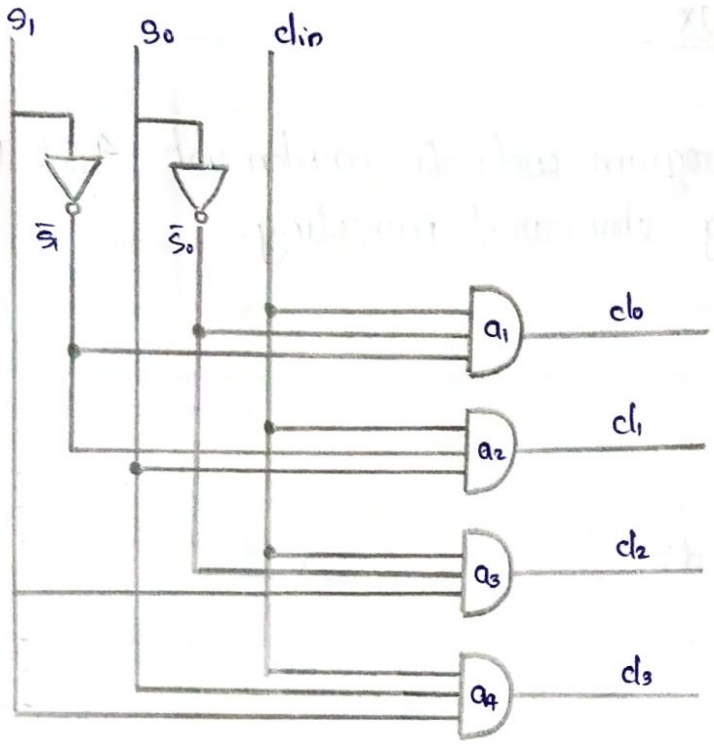
EXPERIMENT-12

DEVELOPMENT OF VERILOG MODULES FOR MUX AND DEMUX

Aim : Develop a verilog program code to implement 4x1 Mux and 1x4 DeMux using structural modeling.

4x1 Mux Program

```
module mux (
 input s0, s1, d0, d1, d2, d3,
 output y
 );
 wire w1, w2, w3, w4;
 not n1 (s0bar, s0);
 not n2 (s1bar, s1);
 and a1 (w1, s1bar, s0bar, d0);
 and a2 (w2, s1bar, s0, d1);
 and a3 (w3, s1, s0bar, d2);
 and a4 (w4, s1, s0, d3);
 or o1 (y, w1, w2, w3, w4);
end module
```

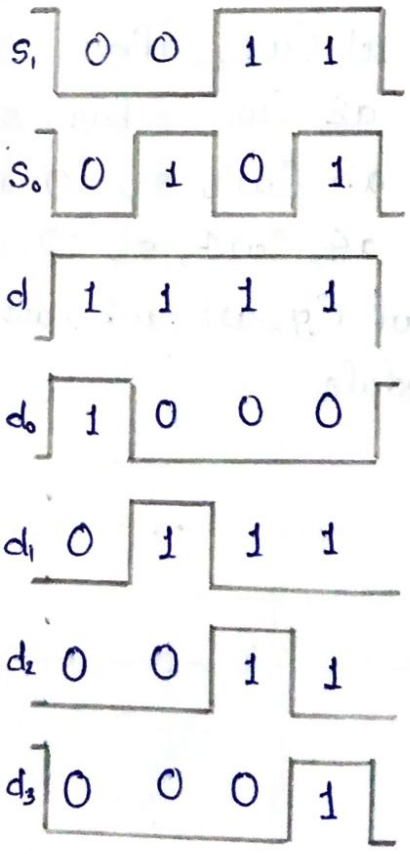


LOGIC DIAGRAM:
1X4 DEMUX

Truth Table:

d_{in}	S_1	S_0	d_0	d_1	d_2	d_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

Waveforms:



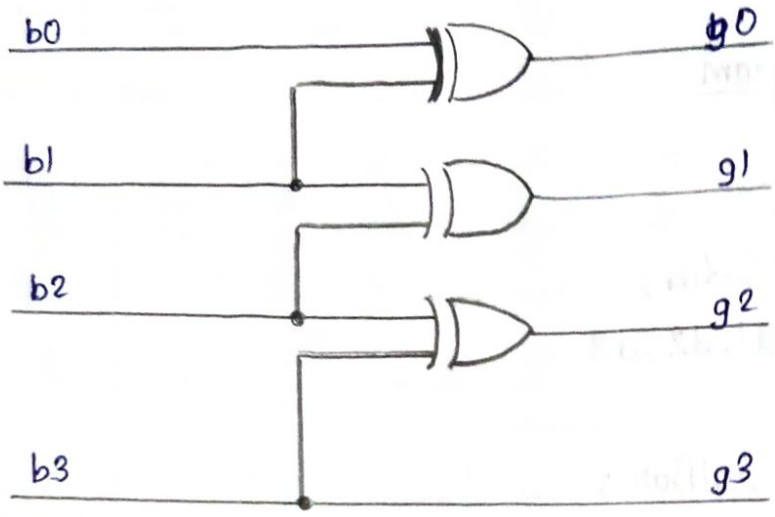
1X4 DeMux Program

```
module demux (
    input s0, s1, din,
    output d0, d1, d2, d3
    );
    wire s0bar, s1bar;
    not n1 (s0bar, s0);
    not n2 (s1bar, s1);
    and a1 (d0, s1bar, s0bar, din);
    and a2 (d1, s1bar, s0, din);
    and a3 (d2, s1, s0bar, din);
    and a4 (d3, s1, s0, din);
endmodule
```

8/8/22

Result :-

Developed a verilog program code to implement 4X1 Mux and 1X4 DeMux using structural modeling.

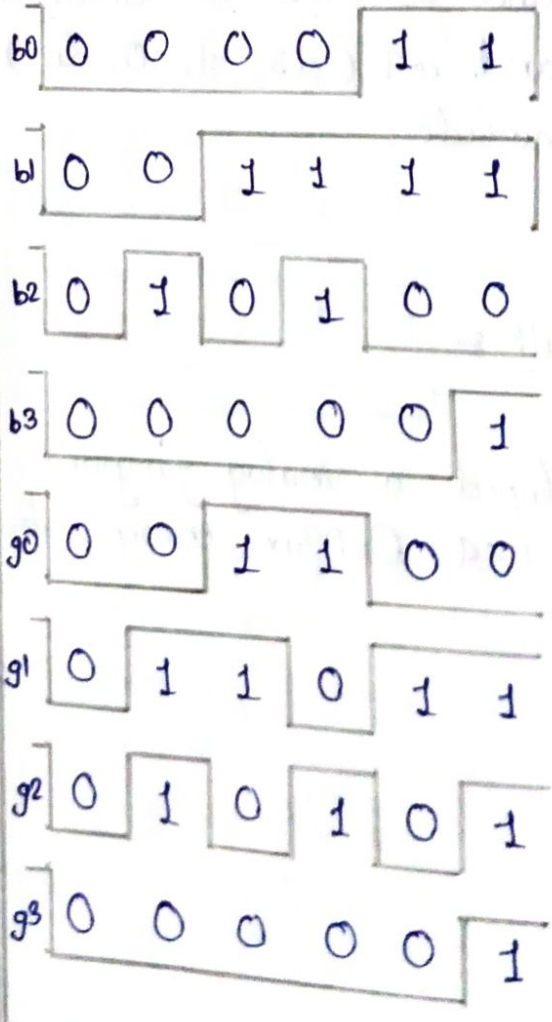


LOGIC DIAGRAM OF BINARY TO GIRAY CODE CONVERTER

Truth Table

b0	b1	b2	b3	g0	g1	g2	g3
0	0	0	0	0	0	0	0
0	0	1	0	0	1	1	0
0	1	0	0	1	1	0	0
0	1	1	0	1	0	1	0
1	1	0	0	0	1	0	0
1	1	0	1	0	1	1	1

Waveforms



08/08/22

EXPERIMENT-13

DEVELOPMENT OF VERILOG MODULE FOR A 4 BIT BINARY TO GREY CODE CONVERTER

Aim : Develop a verilog program code to implement a 4-bit binary to gray code converter.

Program

```
module binarytogrey(
    input b0, b1, b2, b3,
    output g0, g1, g2, g3
    );
    xor x1(g0, b0, b1);
    xor x2(g1, b1, b2);
    xor x3(g2, b2, b3);
    assign g3 = b3;
endmodule
```

Result :-

8/8/22

Developed a verilog program code to implement a 4-bit binary to grey code converter.